by merging the two entries' character sets, and setting its frequency as the sum of the old subtrees' frequencies. Then the new node and two subtrees are combined in a new subtree, where the new node `P` is the root and the subtrees `L` and `R` are its children.

Finally, at line #11 we add this new subtree back into the queue: as it's shown in figure 2.16D, it sometimes can be placed at the top of the queue, but that's not always the case: the priority queue will take care of this detail for us (notice that here the priority queue is used as a black box, as we discussed in section 2.4).

Listing 2.15 The Huffman Coding Algorithm (building a table from the tree)

```
function buildTable(node, sequence, charactersToSequenceMap)
  if node.characters.size == 1
then                                                  #1
    charactersToSequenceMap[node.characters[0]] ← sequence
         #2
  else
         #3
    if node.left <> none
then                                                      #4
      buildTable(node.left, sequence + 0,
charactersToSequenceMap)   #5
    if node.right <> none
then                                             #6
      buildTable(node.right, sequence + 1,
charactersToSequenceMap)  #7
  return
charactersToSequenceMap                                        #8
```